Completado

S1 · Paso 1 de 15

# Preparar OpenSpec en workshop-auth-backend

Instala la CLI, inicializa seleccionando GitHub Copilot, conserva el schema estándar spec-driven y configura el contexto/reglas del laboratorio.

> **Getting started · preparar el workspace**

Prepara OpenSpec para trabajar sobre el backend con GitHub Copilot y el schema spec-driven.

En la raíz de tu workshop-auth-backend local, donde está pom.xml.

1. Comprueba Node.js e instala la CLI.
2. Inicializa el backend con GitHub Copilot seleccionado.
3. Revisa la configuración generada y agrega el contexto del producto sin borrar propiedades existentes.

### Resultado esperado

- El repositorio debe tener la estructura OpenSpec, la integración en .github/ y una configuración que conserve el flujo proposal → specs → design → tasks → apply.

## Comprobar la versión de Node.js

Muestra qué versión de Node.js encuentra tu terminal. OpenSpec necesita Node.js para ejecutar su CLI. En Git Bash de tu equipo, antes de instalar OpenSpec.

Comando

```
node --version
```

## Instalar la CLI de OpenSpec

Descarga e instala OpenSpec para que el comando openspec esté disponible en tu equipo. Esta instrucción prepara la herramienta; la configuración del backend se crea después con openspec init. En Git Bash de tu equipo, con Node.js y npm instalados y acceso al registro de npm.

Comando

```
npm install -g @fission-ai/openspec@latest
```

## Comprobar la instalación de OpenSpec

Consulta la versión de la CLI instalada para confirmar que tu terminal puede encontrar y ejecutar OpenSpec. En Git Bash de tu equipo, dentro del repositorio local del Issue.

Comando

```
openspec --version
```

## Inicializar OpenSpec en el backend

Prepara la estructura de OpenSpec y los workflows de GitHub Copilot dentro de workshop-auth-backend. En tu repositorio local del Issue. En el ejemplo, workshop-auth-backend. Selecciona GitHub Copilot al inicializar OpenSpec.

Comando

```
openspec init
```

## Consultar los cambios de OpenSpec

Lista los cambios activos que OpenSpec reconoce en este repositorio. Sirve para identificar el nombre del cambio que usarás en status y validate. En la raíz de tu repositorio local, después de inicializar OpenSpec.

Comando

```
openspec list
```

Completado

S1 · Paso 2 de 15

# Elegir un Issue real del Project ReqFlow (estado Todo)

Selecciona el issue que guiará el cambio en workshop-auth-backend. Verifica que exista y esté priorizado en el Project.

Selecciona el trabajo aprobado que gobernará el recorrido, con su origen y trazabilidad en ReqFlow.

En Copilot con PM/PO para consultar el Project y en tu backend local para preparar la rama.

1. Consulta el ReqFlow del owner correcto y sus items.
2. Elige un Issue de workshop-auth-backend en Todo que corresponda a un hallazgo confirmado.
3. Reutiliza o crea su rama vinculada y confirma que esté activa.

### Resultado esperado

- Conserva el número y enlace del Issue, su H-xx y el nombre de la rama. Todos los artefactos posteriores deben referirse a ese mismo cambio.

### Si algo no coincide

- Si falta el Issue, la opción No tengo un issue prepara su creación desde HU-01.md.

## Consultar los items de ReqFlow

Recupera el Project ReqFlow del Día 2 y sus items para elegir un Issue existente del backend. En GitHub Copilot Chat de VS Code. Selecciona PM/PO.

Prompt

```
Usando exclusivamente el GitHub MCP oficial, consulta el GitHub Project
"ReqFlow"
del owner <OWNER> y lista sus items: repositorio, título, estado y campo
"Hallazgo origen".

No crees ni modifiques nada todavía.
No uses terminal ni GitHub CLI.
```

## Identificar el Project correcto

Confirma la identidad del Project para que las operaciones siguientes apunten al ReqFlow correcto. En GitHub Copilot Chat de VS Code. Selecciona PM/PO.

Prompt

```
Usando exclusivamente github-official, lista los GitHub Projects de <OWNER> y
confirma el número o identificador del Project ReqFlow.
```

## Entrar en el repositorio del backend

Cambia el directorio de trabajo de la terminal al backend para que las operaciones siguientes se hagan en el repositorio correcto. En Git Bash de tu equipo, partiendo de reqflow-workspace/. Cambia al repositorio del Issue; workshop-auth-backend es el ejemplo de la guía.

Comando

```
cd workshop-auth-backend
```

## Crear la rama vinculada al Issue

Pide a GitHub una rama de desarrollo vinculada al Issue elegido y deja esa rama activa en tu copia local del backend. En el clon local de workshop-auth-backend, con gh autenticado y permisos para crear ramas en ese repositorio.

Comando

```
gh issue develop <ISSUE_NUMBER> --checkout --base main --name <BRANCH_DEL_ISSUE>
```

## Comprobar la rama activa

Muestra el nombre de la rama local actual para confirmar dónde se guardarán tus cambios. En la raíz de tu workshop-auth-backend local, donde está pom.xml.

Comando

```
git branch --show-current
```

**Alternativa para crear el Issue**

Completado

S1 · Paso 3 de 15

# Crear proposal + specs/ desde el Issue

Genera el contrato de cambio (qué se cambia, criterios, no-funcionales) antes de implementar. Pasa por GATE HUMANO antes de continuar a S2.

Define por qué importa el cambio y qué comportamiento se espera antes de decidir cómo implementarlo.

En Copilot con PM/PO, limitado al backend.

1. Consulta el Issue y el contrato documentado.
2. Crea proposal y specs con alcance, fuera de alcance y escenarios observables.
3. Revisa y aprueba los artefactos antes de pasar al diseño.

### Resultado esperado

- El cambio debe referenciar el Issue y especificar qué se preserva o cambia. Todavía no deben añadirse design, tasks ni código funcional en esta etapa.

## Crear proposal y specs desde el Issue

Convierte el Issue aprobado en una propuesta de cambio y escenarios verificables antes de diseñar o implementar. En GitHub Copilot Chat de VS Code. Selecciona PM/PO.

Prompt

```
Trabaja únicamente en workshop-auth-backend.
No leas ni modifiques los otros repositorios del workspace.

Trabaja sobre la rama actual.
No crees ni cambies de rama.

Usa github-official para consultar el GitHub Issue #<ISSUE_NUMBER>
y úsalo como fuente del trabajo comprometido y del alcance.

Trabaja con OpenSpec y el schema estándar spec-driven configurado en este
repositorio.

Usa github-official para consultar el Issue y úsalo como fuente del trabajo
```

```
comprometido y del alcance.
Usa docs/ENDPOINTS.md (contrato documentado el Dia 1) como baseline del
comportamiento que debe preservarse: los escenarios de las specs deben ser
coherentes con ese contrato, o explicar por que el Issue lo cambia.

Crea una propuesta OpenSpec para este cambio y materializa:
- proposal.md;
- las specs necesarias bajo openspec/changes/<change>/specs/.

La propuesta debe:
- referenciar el Issue;
- explicar por qué importa el cambio;
- separar alcance y fuera de alcance;
```

## Consultar los cambios de OpenSpec

Lista los cambios activos que OpenSpec reconoce en este repositorio. Sirve para identificar el nombre del cambio que usarás en status y validate. En la raíz de tu repositorio local, después de inicializar OpenSpec.

Comando

```
openspec list
```

## Revisar el estado de los artefactos

Consulta qué artefactos del cambio están disponibles y cuáles siguen pendientes según el workflow de OpenSpec. En la raíz de tu workshop-auth-backend local, donde está pom.xml.

Comando

```
openspec status --change <CHANGE_NAME>
```

## Validar los artefactos del cambio

Comprueba las reglas de estructura y contenido que OpenSpec aplica a las especificaciones del cambio seleccionado. En la raíz de tu workshop-auth-backend

local, donde está pom.xml.

Comando

```
openspec validate <CHANGE_NAME>
```

Completado

S2 · Paso 4 de 15

# Crear design.md (Plan)

Decisiones técnicas, impacto, estrategia incremental y riesgos.

Convierte las specs aprobadas en decisiones técnicas que permitan un cambio acotado.

En Copilot con Architect, en la misma rama.

1. Lee proposal y specs antes de diseñar.
2. Documenta componentes, contratos, alternativas, riesgos y estrategia incremental.
3. Identifica qué pruebas deben proteger el comportamiento antes del cambio.

### Resultado esperado

- design.md debe explicar las decisiones necesarias para ese Issue y permitir revisar sus riesgos sin ampliar el alcance.

## Diseñar el cambio con Architect

Pide el diseño técnico del cambio a partir de la propuesta y las specs aprobadas. En GitHub Copilot Chat de VS Code. Selecciona Architect.

Prompt

```
Trabaja únicamente en workshop-auth-backend.
No leas ni modifiques los otros repositorios del workspace.
Todos los comandos, lecturas, cambios y operaciones Git/GitHub de este paso
deben referirse únicamente a workshop-auth-backend.

Trabaja sobre la rama actual <BRANCH_DEL_ISSUE>. No crees ni cambies a otra
rama.
```

```
Trabaja sobre el cambio OpenSpec <CHANGE_NAME> ya aprobado.
Lee proposal.md y specs/ antes de proponer cualquier diseño.

Crea o completa:
openspec/changes/<CHANGE_NAME>/design.md

El design debe incluir:
- contexto técnico relevante;
- goals / non-goals;
- contratos protegidos;
- componentes afectados;
- decisiones técnicas y alternativas consideradas;
- estrategia incremental;
- pruebas de caracterización necesarias antes de modificar comportamiento
crítico;
- riesgos / trade-offs;
- rollback cuando aplique.
```

Completado

S2 · Paso 5 de 15

# Crear tasks.md (Tasks)

Checklist de implementación vinculada al Issue. Mantén GitHub como tracking de trabajo comprometido. GATE HUMANO antes de validar.

Ordena el trabajo aprobado en tareas pequeñas y demostrables.

En Copilot con Architect.

1. Deriva las tareas del Issue, proposal, specs y design.
2. Sitúa la caracterización antes del cambio funcional.
3. Revisa que cada tarea tenga una forma de comprobarse.

### Resultado esperado

- tasks.md debe ser coherente con los artefactos y conservar pendientes las tareas que todavía no se ejecutaron.

## Preparar el checklist de tareas

Descompone el diseño aprobado en tareas pequeñas que permitan ejecutar y revisar el cambio. En GitHub Copilot Chat de VS Code. Selecciona Architect.

Prompt

```
Trabaja únicamente en workshop-auth-backend.
No leas ni modifiques los otros repositorios del workspace.
Todos los comandos, lecturas, cambios y operaciones Git/GitHub de este paso deben referirse únicamente a workshop-auth-backend.

Continúa sobre la rama actual <BRANCH_DEL_ISSUE>. No crees ni cambies de rama.

Usa proposal.md, specs/ y design.md ya aprobados del cambio OpenSpec <CHANGE_NAME>.

Genera o completa:
openspec/changes/<CHANGE_NAME>/tasks.md

Las tareas deben:
- estar dentro del alcance del GitHub Issue;
- ser pequeñas y verificables;
- comenzar por las pruebas de caracterización necesarias antes del cambio funcional;
- incluir la implementación incremental;
- incluir pruebas/validaciones relevantes;
- evitar trabajo secundario no aprobado.

El GitHub Issue sigue siendo la unidad de trabajo del producto. tasks.md es el checklist de implementación de este cambio OpenSpec.

No implementes todavía el cambio funcional.
```

Completado

S2 · Paso 6 de 15

# Validar (Validate) antes de tocar código

Ejecuta openspec validate y revisiones humanas de los artefactos (Specify/Plan/Tasks). Recién después sigue el baseline de characterization tests en S4.

Comprueba los artefactos y su coherencia antes de tocar el comportamiento del backend.

En la raíz de tu workshop-auth-backend local, donde está pom.xml.

1. Identifica el nombre real del cambio.
2. Consulta status y ejecuta validate.
3. Revisa proposal, specs, design y tasks con el participante.

### Resultado esperado

- Conserva la validación y los gates humanos de los artefactos. Las pruebas del baseline se realizan después, en S4.

## >_ Consultar los cambios de OpenSpec

Lista los cambios activos que OpenSpec reconoce en este repositorio. Sirve para identificar el nombre del cambio que usarás en status y validate. En la raíz de tu repositorio local, después de inicializar OpenSpec.

>_ Comando

```
openspec list
```

## >_ Revisar el estado de los artefactos

Consulta qué artefactos del cambio están disponibles y cuáles siguen pendientes según el workflow de OpenSpec. En la raíz de tu workshop-auth-backend local, donde está pom.xml.

>_ Comando

```
openspec status --change <CHANGE_NAME>
```

## >_ Validar los artefactos del cambio

Comprueba las reglas de estructura y contenido que OpenSpec aplica a las especificaciones del cambio seleccionado. En la raíz de tu workshop-auth-backend

local, donde está pom.xml.

Comando

```
openspec validate <CHANGE_NAME>
```

Completado

S3 · Paso 7 de 15

# Copiar los Custom Agents al workspace

Descarga los cuatro templates de Agents del portal y guárdalos en reqflow-workspace/.github/agents/, dentro de tu carpeta con matrícula y nombre. Si ya tienes una copia externa de los materiales, puedes copiarlos desde templates/agents/. No necesitas clonar bcp-ai-labs. Conserva los archivos existentes; en S4 paso 0 cópialos también al repositorio del Issue elegido.

Hace disponibles los roles persistentes del taller en el workspace de VS Code.

En reqflow-workspace/.

1. Descarga los cuatro templates de Agents desde los enlaces de la guía del portal. Si ya tienes una copia externa de los materiales, puedes usar templates/agents/ de esa copia.
2. Guarda pm-po.agent.md, discovery.agent.md, architect.agent.md y developer.agent.md en .github/agents/ del workspace, conservando archivos existentes.
3. Refresca VS Code y comprueba que puedes seleccionar los perfiles.

### Resultado esperado

- Debe existir reqflow-workspace/.github/agents/ con los cuatro perfiles. La preparación del módulo SDD también pide copiarlos al repositorio del Issue elegido.

Completado

S3 · Paso 8 de 15

# Probar los límites de un agente aislado

Selecciona un solo Custom Agent (Discovery, PM/PO, Architect o Developer) sin usar tools y verifica que explique su responsabilidad y a qué rol debe pasar lo que no le corresponde.

Prueba las responsabilidades de cada agente de forma aislada antes de usar sus tools.

En Copilot Chat, seleccionando un solo Custom Agent por prueba.

1. Abre el prompt correspondiente al rol elegido.
2. Ejecútalo sin lecturas de archivos ni tools, como indica el texto.
3. Revisa el límite que explica el agente y el rol al que deriva lo que no le corresponde.

### Resultado esperado

- Debes poder justificar qué decide cada rol y comprobar que respeta el límite solicitado en la prueba.

## Probar los límites de Discovery

Comprueba que Discovery reconoce su límite antes de darle acceso al producto. En GitHub Copilot Chat de VS Code. Selecciona Discovery.

Prompt

```
Esta es únicamente una prueba de los límites de tu rol.
No leas archivos, no busques en el workspace y no uses MCP ni ninguna otra tool.

Supón que durante un análisis encuentras un problema importante en ReqFlow.
¿Puedes corregirlo directamente?
Explica qué deberías hacer según tu responsabilidad antes de que alguien
modifique código.
```

## Probar los límites de PM/PO

Comprueba que PM/PO separa la decisión de producto del diseño y la implementación técnica. En GitHub Copilot Chat de VS Code. Selecciona PM/PO.

Prompt

```
Esta es únicamente una prueba de los límites de tu rol.
No leas archivos y no uses ninguna tool.

Supón que se detectó deuda en la capa de persistencia de ReqFlow.
¿Debes decidir y ejecutar tú mismo el refactor técnico?
Explica qué te corresponde hacer y a qué rol debería pasar la decisión técnica.
```

## Probar los límites de Architect

Comprueba que Architect define decisiones técnicas sin asumir la implementación del cambio. En GitHub Copilot Chat de VS Code. Selecciona Architect.

Prompt

```
Esta es únicamente una prueba de los límites de tu rol.
No leas archivos y no uses ninguna tool.

Supón que recibes un hallazgo técnico ya confirmado.
¿Debes implementar directamente el cambio en código?
Explica cuál es tu responsabilidad y cuándo debe intervenir Developer.
```

## Probar los límites de Developer

Comprueba que Developer pide trabajo aprobado antes de modificar el producto. En GitHub Copilot Chat de VS Code. Selecciona Developer.

Prompt

```
Esta es únicamente una prueba de los límites de tu rol.
No leas archivos y no uses ninguna tool.

Te piden mejorar todo ReqFlow sin issue, spec ni alcance aprobado.
¿Debes empezar a modificar código?
Explica qué información necesitas antes de implementar.
```

Completado

S3 · Paso 9 de 15

# Verificar MCP oficial + Projects habilitado

Confirma que github-official expone tools de Projects y que puedes leer Projects vía MCP.

Comprueba que VS Code puede consultar el backend y Projects a través del MCP oficial.

En VS Code con Agent estándar y github-official.

1. Revisa .vscode/mcp.json y el toolset projects.
2. Comprueba la conexión y las tools disponibles en Configure Tools.
3. Ejecuta las consultas de lectura y conserva la tool y el resultado.

### Resultado esperado

- La evidencia debe mostrar una lectura real de Projects y del repositorio mediante MCP. Si hay un problema de permisos, resuélvelo antes de operar el Project.

## Comprobar Projects mediante MCP

Verifica que el MCP oficial puede consultar Projects con tu identidad y permisos actuales. En GitHub Copilot Chat de VS Code. Selecciona Agent estándar.

Prompt

```
Usando exclusivamente github-official, lista los GitHub Projects accesibles para mi usuario.

No uses terminal, GitHub CLI, web ni otras herramientas.
Indica qué tool MCP utilizaste.
```

## Consultar el backend mediante MCP

Verifica una lectura del repositorio del backend mediante GitHub MCP. En GitHub Copilot Chat de VS Code. Selecciona Agent estándar.

Prompt

```
Usando exclusivamente github-official, consulta información de <OWNER>/workshop-auth-backend y dime qué tool MCP utilizaste.
No uses terminal ni GitHub CLI para obtener la información de GitHub.
```

Completado

S3 · Paso 10 de 15

# Completar backlog (sin inventar trabajo)

Discovery revalida HALLAZGOS.md y convierte hallazgos revisados en issues reales y verificables en el repo correcto.

Completa el backlog con hallazgos revisados y trazables, aprovechando los issues de los días anteriores.

Discovery analiza; PM/PO crea o incorpora trabajo aprobado.

1. Revalida HALLAZGOS.md contra código, contratos y pruebas actuales.
2. Clasifica humanamente los hallazgos.
3. Reutiliza issues existentes; crea los que falten en el repositorio correcto y agrégalos a ReqFlow.

## Resultado esperado

- Cada Issue debe tener evidencia, aceptación verificable e Id H-xx. El Project debe conservar esa trazabilidad y evitar duplicados.

## Revalidar los hallazgos del backend

Revalida los hallazgos del backend contra su estado actual para construir backlog con evidencia. En GitHub Copilot Chat de VS Code. Selecciona Discovery.

Prompt

```
Analiza workshop-auth-backend sin modificar archivos.

Parte de lo que ya existe: workshop-auth-backend/HALLAZGOS.md (Dia 1) y
los issues del Project ReqFlow (Dia 2). Re-verifica cada hallazgo "Pendiente"
contra el codigo actual e indica si ya tiene issue (campo Hallazgo origen).
Despues comprende estructura, contratos (docs/ENDPOINTS.md) y pruebas
existentes (docs/TESTING.md).
Después presenta como máximo 5 hallazgos candidatos.

Para cada hallazgo incluye:
- evidencia concreta;
- por qué importa;
- prioridad propuesta;
- contrato funcional que podría verse afectado;
- cómo lo validarías antes de cambiarlo.

No inventes problemas si no encuentras evidencia.
No implementes soluciones.
```

## Crear issues desde hallazgos confirmados

Convierte únicamente hallazgos aprobados en issues trazables en su repositorio correcto. En GitHub Copilot Chat de VS Code. Selecciona PM/PO.

Prompt

```
Usando exclusivamente github-official, convierte estos hallazgos CONFIRMADOS en
issues.

Reglas:
- cada issue debe vivir en el repositorio al que pertenece la evidencia;
```

```
- no dupliques: si el hallazgo ya tiene issue en ReqFlow, reutilízalo;
- incluye el Id H-xx del hallazgo de origen y completa el campo Hallazgo origen;
- no inventes nuevos hallazgos;
- incluye problema, evidencia, impacto y criterio verificable;
- antes de crear, muéstrame el repositorio destino y el título propuesto.
```

### Agregar los issues aprobados a ReqFlow

Añade al Project ReqFlow los issues que ya aprobaste para coordinarlos desde el producto. En GitHub Copilot Chat de VS Code. Selecciona PM/PO.

Prompt

```
Usando exclusivamente las tools de Projects de github-official, agrega al
Project ReqFlow los issues que acabamos de aprobar.
No cambies su repositorio de origen.
```

Completado

S3 · Paso 11 de 15

# De single-agent a routing

Mapea cada rol a un tipo de trabajo: producto/alcance → PM/PO, hallazgo → Discovery, diseño → Architect, implementación → Developer. Prepara el routing explícito de S4.

Define a qué rol se entrega cada tipo de trabajo y qué necesita recibir para empezar.

En Copilot, eligiendo explícitamente el Agent adecuado.

1. Producto y alcance se dirigen a PM/PO.
2. Hallazgos se dirigen a Discovery; diseño, a Architect.
3. Implementación se dirige a Developer después de las aprobaciones.

### Resultado esperado

- Debes explicar el motivo de cada traspaso y sus entradas. El laboratorio usa selección explícita de roles; no pide construir un router autónomo.

Completado

S4 · Paso 12 de 15

# Confirmar el routing del Issue

Verifica que proposal/specs/design/tasks ya pasaron por PM/PO y Architect antes de que Developer implemente.

Confirma que el trabajo llega a Developer con el alcance y el plan aprobados.

En el Issue, el cambio OpenSpec y la rama del backend.

1. Revisa el vínculo entre Issue, proposal, specs, design y tasks.
2. Confirma la revisión de PM/PO y Architect.
3. Valida el cambio antes de comenzar las tareas de caracterización.

### Resultado esperado

- Debe existir una cadena de decisiones revisable y un plan aprobado para el mismo Issue. La validación de OpenSpec se acompaña de la revisión humana.

### >_ Validar los artefactos del cambio

Comprueba las reglas de estructura y contenido que OpenSpec aplica a las especificaciones del cambio seleccionado. En la raíz de tu workshop-auth-backend local, donde está pom.xml.

>_ Comando

```
openspec validate <CHANGE_NAME>
```

Completado

S4 · Paso 13 de 15

# Proteger baseline (characterization)

Asegura que el comportamiento actual quede cubierto antes de implementar.

Obtiene una línea base de comportamiento que permita detectar regresiones durante la implementación.

En Copilot con Developer y Maven en el backend.

1. Revisa las pruebas ya documentadas.
2. Añade solo la caracterización que falte para el alcance aprobado.
3. Ejecuta las pruebas, registra sus resultados y actualiza únicamente las tareas verificadas.

### Resultado esperado

- Conserva el comando, el resultado y el comportamiento protegido por cada prueba. El diff de esta etapa debe limitarse a las pruebas necesarias y tasks.md.

## Caracterizar el comportamiento existente

Protege el comportamiento existente del backend antes de implementar el cambio funcional. En GitHub Copilot Chat de VS Code. Selecciona Developer.

Prompt

```
Trabaja únicamente en workshop-auth-backend.
No leas ni modifiques los otros repositorios del workspace.
Todos los comandos, lecturas, cambios y operaciones Git/GitHub de este paso deben referirse únicamente a workshop-auth-backend.

Trabaja sobre la rama actual vinculada al GitHub Issue. No crees ni cambies de rama.

Lee el GitHub Issue y el cambio OpenSpec <CHANGE_NAME> completo:
- proposal.md;
- specs/;
- design.md;
- tasks.md.

Todavía no implementes el cambio funcional.

Ejecuta únicamente las tareas de tasks.md relacionadas con caracterización y protección del comportamiento existente.

Antes de crear pruebas nuevas, revisa docs/TESTING.md (Dia 1): reutiliza las que ya protegen el comportamiento y agrega solo las que falten.

Crea las pruebas de caracterización necesarias y ejecútalas.
```

```
Reglas:
- no corrijas todavía el problema objeto del Issue;
```

### Ejecutar las pruebas del backend

Usa Maven y el pom.xml del workshop para compilar el código y ejecutar las pruebas Java del proyecto. En la raíz de tu workshop-auth-backend local, donde está pom.xml.

Comando

```
mvn test
```

Completado

S4 · Paso 14 de 15

# Aplicar e implementar incrementalmente

Usa OpenSpec apply, implementa, ejecuta tests y evidencia.

Implementa el Issue aprobado con OpenSpec apply y evidencia incremental.

En Copilot con Developer, sobre la misma rama.

1. Lee los artefactos aprobados y el baseline.
2. Ejecuta las tareas funcionales pendientes en cambios pequeños.
3. Prueba los cambios y actualiza el checklist cuando exista evidencia.

### Resultado esperado

- El diff debe permanecer dentro del Issue, las pruebas relevantes deben tener resultados registrados y cada check debe corresponder a trabajo demostrado.

### Implementar las tareas aprobadas

Ejecuta el cambio aprobado usando OpenSpec apply y el checklist de tareas, con la caracterización como red de seguridad. En GitHub Copilot Chat de VS Code. Selecciona Developer.

Prompt

```
Trabaja únicamente en workshop-auth-backend.
No leas ni modifiques los otros repositorios del workspace.
Todos los comandos, lecturas, cambios y operaciones Git/GitHub de este paso
deben referirse únicamente a workshop-auth-backend.

Trabaja sobre la rama actual vinculada al GitHub Issue. No crees ni cambies de
rama.

Implementa el GitHub Issue seleccionado utilizando el cambio OpenSpec
<CHANGE_NAME> como definición y plan aprobados.

Fuentes de verdad:
1. GitHub Issue aprobado;
2. openspec/changes/<CHANGE_NAME>/proposal.md;
3. openspec/changes/<CHANGE_NAME>/specs/;
4. openspec/changes/<CHANGE_NAME>/design.md;
5. openspec/changes/<CHANGE_NAME>/tasks.md;
6. pruebas de caracterización existentes y su baseline.

Usa el workflow OpenSpec apply disponible para GitHub Copilot.

Reglas:
- implementa siguiendo las tareas pendientes de tasks.md;
- marca una tarea como completada solo cuando exista evidencia de que terminó;
- no amplíes el scope fuera del Issue y OpenSpec;
- no cambies contratos fuera del alcance;
```

Completado

S4 · Paso 15 de 15

# Evaluator + DoD + PR

Ejecuta Definition of Done, abre PR con Closes #N y revisa checks. GATE HUMANO de cierre antes de integrar a main.

Prepara una revisión independiente y decide el cierre usando evidencia del cambio.

En Copilot con Developer, GitHub y revisión del participante.

1. Valida los artefactos y revisa el diff y las tareas.
2. Prepara el PR de la rama del Issue hacia main con Closes #N y evidencia real.
3. Evalúa DoD, revisa los checks y toma la decisión humana antes de integrar.

### Resultado esperado

- El PR debe permitir reconstruir Issue → artefactos → pruebas → implementación. Cierra solo los criterios demostrados y registra los que tengan falta de evidencia o bloqueos.

## Validar los artefactos del cambio

Comprueba las reglas de estructura y contenido que OpenSpec aplica a las especificaciones del cambio seleccionado. En la raíz de tu workshop-auth-backend local, donde está pom.xml.

Comando

```
openspec validate <CHANGE_NAME>
```

## Preparar el contenido del PR

Prepara un Pull Request que un revisor pueda evaluar con la evidencia del cambio.
En GitHub Copilot Chat de VS Code. Selecciona Developer.

Prompt

```
Trabaja únicamente en workshop-auth-backend.
No leas ni modifiques los otros repositorios del workspace.
Todos los comandos, lecturas, cambios y operaciones Git/GitHub de este paso
deben referirse únicamente a workshop-auth-backend.

Prepara el Pull Request final del GitHub Issue desde la rama actual
<BRANCH_DEL_ISSUE> hacia main.

Usa únicamente evidencia existente en:
- el GitHub Issue;
- openspec/changes/<CHANGE_NAME>/proposal.md;
- openspec/changes/<CHANGE_NAME>/specs/;
- openspec/changes/<CHANGE_NAME>/design.md;
- openspec/changes/<CHANGE_NAME>/tasks.md;
```

```
- diff acumulado;
- pruebas ejecutadas.

Antes de preparar el PR:
- verifica que estás trabajando con <BRANCH_DEL_ISSUE>;
- revisa que tasks.md refleje el trabajo realmente completado;
- revisa el diff acumulado;
- confirma que las pruebas relevantes tienen evidencia.

Prepara:
- título;
- descripción;
```

## Evaluar la Definition of Done

Evalúa la Definition of Done con evidencia existente para decidir si el cambio puede cerrarse. En GitHub Copilot Chat de VS Code. Selecciona Developer.

Prompt

```
Trabaja únicamente en workshop-auth-backend.
No leas ni modifiques los otros repositorios del workspace.
Todos los comandos, lecturas y operaciones Git/GitHub de esta evaluación deben
referirse únicamente a workshop-auth-backend.

Usa el Skill reqflow-definition-of-done para evaluar el cambio actual.

Evalúa únicamente con evidencia existente en:
- el GitHub Issue;
- el Pull Request final;
- openspec/changes/<CHANGE_NAME>/proposal.md;
- openspec/changes/<CHANGE_NAME>/specs/;
- openspec/changes/<CHANGE_NAME>/design.md;
- openspec/changes/<CHANGE_NAME>/tasks.md;
- pruebas ejecutadas;
- checks disponibles en GitHub.

Para cada criterio clasifica el resultado como:
```

```
- CUMPLIDO;
- PENDIENTE DE OTRO GATE;
- NO APLICA AL ALCANCE;
- FALTA EVIDENCIA;
- BLOQUEADO.

No declares un criterio como cumplido si no existe evidencia verificable.
No modifiques código ni archivos.
```